

CS Mini-Projects Technical Sheets

Quanta Mini-Projects Team

February 2025

Contents

1	Mini-Project One : Quanta AI Chatbot	7
1.1	Description	7
1.2	Objectives	7
1.3	Technical Requirements	7
1.4	Prerequisites	7
1.5	Difficulty Level	8
1.6	Deliverables	8
1.7	Team Size	8
2	Mini-Project Two : Game Development	9
2.1	Description	9
2.2	Objectives	9
2.3	Technical Requirements	9
2.4	Prerequisites	9
2.5	Difficulty Level	10
2.6	Deliverables	10
2.7	Team Size	10
3	Mini-Project Three : Smart Home Simulations	11
3.1	Description	11
3.2	Objectives	11
3.3	Technical Requirements	11
3.4	Prerequisites	11
3.5	Difficulty Level	12
3.6	Deliverables	12
3.7	Team Size	12
4	Mini-Project Four : Face detection	13
4.1	Description	13
4.2	Objectives	13
4.3	Technical Requirements	13
4.4	Prerequisites	13

4.5	Difficulty Level	14
4.6	Deliverables	14
4.7	Team Size	14
5	Mini-Project Five : sorting algorithm visualizer	15
5.1	Description	15
5.2	Objectives	15
5.3	Technical Requirements	15
5.4	Prerequisites	15
5.5	Difficulty Level	16
5.6	Deliverables	16
5.7	Team Size	16
6	Mini-Project Six : Password Manager	17
6.1	Description	17
6.2	Objectives	17
6.3	Technical Requirements	17
6.4	Prerequisites	17
6.5	Difficulty Level	17
6.6	Deliverables	18
6.7	Team Size	18
7	Mini-Project Seven : Voice Assistant	19
7.1	Description	19
7.2	Objectives	19
7.3	Technical Requirements	19
7.4	Prerequisites	19
7.5	Difficulty Level	19
7.6	Deliverables	19
7.7	Team Size	20
8	Mini-Project Eight : Personal budget tracker App	21
8.1	Description	21
8.2	Objectives	21
8.3	Technical Requirements	21
8.4	Prerequisites	21
8.5	Difficulty Level	21
8.6	Deliverables	21
8.7	Team Size	22
9	Mini-Project Nine : Blockchain Basics	23
9.1	Description	23
9.2	Objectives	23
9.3	Technical Requirements	23
9.4	Prerequisites	23
9.5	Difficulty Level	23

9.6	Deliverables	23
9.7	Team Size	24
10	Mini-Project Ten : AI Image classifier	25
10.1	Description	25
10.2	Objectives	25
10.3	Technical Requirements	25
10.4	Prerequisites	25
10.5	Difficulty Level	25
10.6	Deliverables	25
10.7	Team Size	26
11	Mini-Project Eleven : Interactive story game	27
11.1	Description	27
11.2	Objectives	27
11.3	Technical Requirements	27
11.4	Prerequisites	27
11.5	Difficulty Level	27
11.6	Deliverables	27
11.7	Team Size	28
12	Mini-Project Twelve : Data Encryption Tool	29
12.1	Description	29
12.2	Objectives	29
12.3	Technical Requirements	29
12.4	Prerequisites	29
12.5	Difficulty Level	29
12.6	Deliverables	29
12.7	Team Size	30
13	Mini-Project Thirteen : Augmented Reality	31
13.1	Description	31
13.2	Objectives	31
13.3	Technical Requirements	31
13.4	Prerequisites	31
13.5	Difficulty Level	31
13.6	Deliverables	32
13.7	Team Size	32
14	Mini-Project Fourteen : Chat Application	33
14.1	Description	33
14.2	Objectives	33
14.3	Technical Requirements	33
14.4	Prerequisites	33
14.5	Difficulty Level	33
14.6	Deliverables	34

14.7 Team Size	34
15 Mini Project Fifteen : Personal Portfolio Website Development	35
15.1 Description	35
15.2 Objectives	35
15.3 Technical Requirements	35
15.4 Prerequisites	35
15.5 Difficulty Level	35
15.6 Deliverables	35
15.7 Team Size	36
16 Mini Project Sixteen : Weather Forecast Mobile App Project	37
16.1 Description	37
16.2 Objectives	37
16.3 Technical Requirements	37
16.4 Prerequisites	37
16.5 Difficulty Level	38
16.6 Deliverables	38
16.7 Team Size	38
17 Mini Project Seventeen : Stock Price Predictor	39
17.1 Description	39
17.2 Objectives	39
17.3 Technical Requirements	39
17.4 Prerequisites	39
17.5 Difficulty Level	40
17.6 Deliverables	40
17.7 Team Size	40
18 Mini Project Eighteen : Collaborative Coding Platform	41
18.1 Description	41
18.2 Objectives	41
18.3 Mini Project Nineteen : Technical Requirements	41
18.4 Prerequisites	41
18.5 Difficulty Level	42
18.6 Deliverables	42
18.7 Team Size	42
19 Mini Project Nineteen : Cloud Storage System	43
19.1 Description	43
19.2 Objectives	43
19.3 Technical Requirements	43
19.4 Prerequisites	43
19.5 Difficulty Level	44
19.6 Deliverables	44

19.7 Team Size	44
20 Mini Project Twenty : Check-In System	45
20.1 Description	45
20.2 Objectives	45
20.3 Technical Requirements	45
20.4 Prerequisites	45
20.5 Difficulty Level	46
20.6 Deliverables	46
20.7 Team Size	46
21 Mini Project Twenty-One : Alarm Mobile App Project	47
21.1 Description	47
21.2 Objectives	47
21.3 Technical Requirements	47
21.4 Prerequisites	47
21.5 Difficulty Level	47
21.6 Deliverables	48
21.7 Team Size	48
22 Mini Project Twenty-Two : AI Virtual Assistant for Scheduling	49
22.1 Description	49
22.2 Objectives	49
22.3 Technical Requirements	49
22.4 Prerequisites	49
22.5 Difficulty Level	50
22.6 Deliverables	50
22.7 Team Size	50
23 Mini Project Twenty-Three : AI for Language Translation	51
23.1 Description	51
23.2 Objectives	51
23.3 Technical Requirements	51
23.4 Prerequisites	51
23.5 Difficulty Level	52
23.6 Deliverables	52
23.7 Team Size	52
24 Mini Project Twenty-Four : 3D Interactive Web Showcase	53
24.1 Description	53
24.2 Objectives	53
24.3 Technical Requirements	53
24.4 Prerequisites	53
24.5 Difficulty Level	54
24.6 Deliverables	54

24.7 Team Size	54
--------------------------	----

1 Mini-Project One : Quanta AI Chatbot

1.1 Description

Develop an AI-powered chatbot to assist visitors on Quanta club's website. The chatbot will provide automated responses to frequently asked questions (FAQs) about Quanta Club, including the club's mission, hierarchy, membership, ongoing projects, upcoming events, and guiding users to relevant resources.

The chatbot will be integrated into the Quanta Club website and built using Python with NLP libraries like NLTK. It may include text-based interaction or quick-access buttons for common queries.

Future enhancements could include multilingual support, quizzes, and event reminders for improved user engagement.

1.2 Objectives

- **Automate FAQ Responses** – Provide instant and accurate answers to common questions about Quanta Club.
- **Guide Users to Relevant Resources** – Direct visitors to specific pages or documents related to their queries.
- **Enable Easy Maintenance & Updates** – Implement a simple method for admins to update FAQs as club activities evolve.
- **Ensure Smooth Website Integration** – Embed the chatbot seamlessly into the Quanta Club website for a user-friendly experience.
- **Support Future Enhancements** – Design the chatbot with scalability in mind for potential features like multilingual support, quizzes, and event reminders.

1.3 Technical Requirements

Software : Python (3.7+), Flask/Django, NLTK or spaCy, Git, Database: SQLite or Firebase (for storing FAQs and updates)

Hardware : Computer , Internet connection

Other : Access to website source code for integration

1.4 Prerequisites

- **Python & NLP Basics** – Understanding Python fundamentals and NLP concepts like tokenization and text preprocessing.
- **Database Management** – Basic knowledge of SQLite, Firebase, or other databases for storing FAQs.

- Web & Backend Development – Familiarity with Flask/Django for API development and backend integration.
- Frontend & UI Integration – Experience with HTML, CSS, JavaScript, and the existing website framework (Next.js) for embedding the chatbot.

1.5 Difficulty Level

Intermediate

1.6 Deliverables

- Chatbot System – A functional AI chatbot answering FAQs about Quanta Club.
- Source Code Repository – A well-documented repository (e.g., GitHub) with all code, scripts, and configurations.
- FAQ Database & Management System – A structured FAQ database with a simple interface or script for admins to update FAQs.
- Website Integration – Embedded chatbot within the Quanta Club website for user interaction.
- Technical & User Documentation – Guides covering system architecture, setup, usage, and future improvements.

1.7 Team Size

Maximum: 4 members

2 Mini-Project Two : Game Development

2.1 Description

Develop a basic 2D game using Unity (C#) or Pygame (Python) for Desktop. The game will be single-player, with the specific genre left open to allow creative freedom.

The project will cover core game development principles, including game loops, player controls, UI elements, and collision handling. The final game should have a functional gameplay mechanic, a simple UI, and a structured codebase.

Future improvements may include animations, sound effects, score tracking, and additional levels to enhance the user experience.

2.2 Objectives

- Implement Core Game Mechanics – Ensure smooth player controls, collision detection, and game logic.
- Develop a Functional UI – Include basic menus, score tracking (if applicable), and game over screens.
- Encourage Creativity – Members design their own game mechanics and assets.
- Allow for Future Enhancements – Design the game with the possibility of adding animations, sound effects, or new levels.

2.3 Technical Requirements

Software : Unity (C#) or Pygame (Python) , Visual Studio (for Unity) / PyCharm or VSCode (for Pygame), Git (for version control)

Hardware : PC with decent specifications, Internet connection

Other : Access to game assets (free sprites, sounds, or self-made assets)

2.4 Prerequisites

- Programming Skills – Familiarity with C# (for Unity) or Python (for Pygame).
- Game Development Basics – Understanding game loops, rendering, and input handling.
- Graphics & UI Basics – Experience with sprite handling, animations, and UI elements.
- Version Control – Basic knowledge of Git for collaboration and project management.

2.5 Difficulty Level

Intermediate

2.6 Deliverables

- Playable 2D Game – A working single-player game with interactive mechanics.
- Source Code Repository – A well-documented GitHub repository with code and assets.
- Basic Game Assets – Sprites, background, and sounds (either custom-made or sourced).
- Game Instructions & Documentation – A guide explaining game mechanics, controls, and setup.

2.7 Team Size

Maximum: 3 members

3 Mini-Project Three : Smart Home Simulations

3.1 Description

Create a simulation of a smart home system using Arduino boards, sensors, and actuators (such as lights or fans). The system will use sensors like temperature, motion, or light to trigger the activation or deactivation of devices like lights or fans. The project will demonstrate basic automation and control mechanisms that are foundational in smart home technologies.

The goal is to design and program an Arduino-based system where sensors detect environmental changes and trigger corresponding actions (such as turning lights on/off or controlling fans). The project will also incorporate the use of relays, sensors, and basic logic to create the simulation of a smart home environment.

3.2 Objectives

- Automate Home Appliances: Use sensors to detect changes in the environment (e.g., motion, temperature, light) and control appliances such as lights or fans.
- Program and Interface Sensors with Arduino: Program Arduino to interact with various sensors and control devices accordingly.
- Simulate a Smart Home Environment: Create a system where the appliances are controlled automatically based on sensor input, simulating a smart home environment.
- Ensure System Reliability: Ensure that sensors and appliances work reliably and respond correctly to environmental changes.

3.3 Technical Requirements

Software : Arduino IDE, Fritzing (for circuit design), Git (for version control)

Hardware : Arduino board (e.g., Arduino Uno), sensors (temperature sensor, motion sensor, light sensor), actuators (LEDs, fans, relays), jumper wires, breadboard

Other : Power supply for Arduino, optional: additional components for more complex sensor integration (e.g., humidity sensor, PIR motion sensor)

3.4 Prerequisites

- Basic Programming Skills: Familiarity with C++ for Arduino programming.

- **Hardware Understanding:** Understanding how to wire sensors and actuators to an Arduino.
- **Sensor Integration:** Familiarity with integrating sensors such as temperature, motion, or light sensors into an Arduino circuit.
- **Basic Electronics:** Ability to work with basic electrical components such as LEDs, relays, and sensors.

3.5 Difficulty Level

Intermediate.

3.6 Deliverables

- **Functional Smart Home Simulation:** A working smart home system that controls appliances based on sensor input.
- **Source Code Repository:** A well-documented GitHub repository containing the Arduino code and any additional files.
- **Wiring Diagram & Schematics:** A clear schematic of the circuit design used to control the appliances.
- **Project Documentation:** A guide explaining how the system works, including setup instructions, sensor configurations, and usage.

3.7 Team Size

Maximum: 3 members

4 Mini-Project Four : Face detection

4.1 Description

Develop a face detection system using OpenCV to detect and highlight human faces in images or real-time video streams. The project will focus on implementing a basic face detection algorithm using Haar cascades or deep learning-based models (e.g., DNN, HOG + SVM, or CNN-based models like MTCNN).

The system should be able to detect human faces from a live webcam feed or pre-loaded images, draw bounding boxes around detected faces. Optionally, allow future improvements such as face recognition, emotion detection, or tracking.

4.2 Objectives

- **Implement Face Detection:** Use OpenCV to detect faces in real-time video or images.
- **Optimize Performance:** Ensure efficient and accurate detection with minimal false positives.
- **Allow Future Enhancements:** Structure the codebase to support future improvements like face recognition or emotion analysis.
- **Provide a Simple UI (Optional):** Display detection results with bounding boxes around detected faces.

4.3 Technical Requirements

Software : Python, OpenCV, NumPy, Jupyter Notebook / PyCharm / VS Code

Hardware : PC with a webcam (for real-time detection)

Other : Pre-trained Haar cascade or deep learning models for face detection

4.4 Prerequisites

- **Python Programming:** Familiarity with Python syntax and libraries.
- **Image Processing Basics:** Understanding of computer vision concepts like edge detection, feature extraction, and image filtering.
- **Machine Learning/AI Fundamentals (Optional):** Knowledge of how deep learning models can improve face detection.

4.5 Difficulty Level

Intermediate

4.6 Deliverables

- Face Detection System: A working program that detects and highlights faces in images or video.
- Source Code Repository: A well-documented GitHub repository with code and dependencies.
- Dataset & Model Files: Pre-trained models (Haar cascades, deep learning models if used).
- Project Documentation: A guide explaining the setup, implementation, and usage of the system.

4.7 Team Size

Maximum: 2 members

5 Mini-Project Five : sorting algorithm visualizer

5.1 Description

This project aims to develop an interactive tool that visually demonstrates how different sorting algorithms work. The tool will animate the sorting process in real-time, allowing users to observe how elements are arranged step by step. Users will be able to choose from multiple sorting algorithms and adjust parameters like array size and speed.

The visualization will help learners understand algorithm efficiency and behavior through graphical representation. Future enhancements could include performance comparisons, user-controlled sorting speed, and additional sorting techniques.

5.2 Objectives

- Implement Multiple Sorting Algorithms: Support at least 3-5 algorithms such as Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort.
- Create a Visual Representation: Use graphical bars or other visual elements to represent array elements.
- Enable User Interaction: Allow users to choose an algorithm, set array size, and control speed.
- Optimize Performance: Ensure smooth animations without lag.

5.3 Technical Requirements

Software : Python (Pygame, Tkinter, Matplotlib) or JavaScript (HTML, CSS, JavaScript with Canvas/SVG) , GIT

Hardware : PC with a modern web browser or Python environment

5.4 Prerequisites

- Programming Skills: Knowledge of Python or JavaScript
- Algorithm Understanding: Familiarity with sorting algorithms and their complexities
- Basic UI/Graphics Experience: Working with Pygame, Tkinter, or JavaScript Canvas for visual representation

5.5 Difficulty Level

easy to Intermediate

5.6 Deliverables

- Sorting Visualizer Tool: A functional and interactive visualization of sorting algorithms.
- Source Code Repository: A well-documented GitHub repository with all code and assets.
- Doc & User Guide: Documentation explaining the project setup, implementation and Instructions on how to use the tool and understand the visualized algorithms.

5.7 Team Size

Maximum: 1 member

6 Mini-Project Six : Password Manager

6.1 Description

This project involves developing a basic password storage and encryption tool that securely stores user credentials. The system will allow users to save, retrieve, and manage passwords while ensuring strong encryption for security. The tool can be a desktop application (GUI-based) or a command-line program.

6.2 Objectives

- Secure Password Storage – Encrypt and store passwords in a secure database or file.
- User Authentication – Optionally implement a master password to access stored credentials.
- Encryption & Decryption – Use cryptographic methods (AES, SHA-256, or similar) to protect passwords.
- User-Friendly Interface – Provide an easy way to add, retrieve, and delete passwords.

6.3 Technical Requirements

Software : Python (Tkinter for GUI), Cryptography library, SQLite (for secure storage), VS Code / PyCharm / GIT

Hardware : PC with standard specifications

6.4 Prerequisites

- Knowledge of cryptography and hashing algorithms
- Basic file handling or database management
- Familiarity with Python, C++, or Java for backend logic
- UI development skills for a graphical interface

6.5 Difficulty Level

easy to Intermediate

6.6 Deliverables

- Final Product: A working password manager with encryption
- Source Code Repository: Well-documented GitHub repo
- Encryption Implementation: Securely stored passwords
- Project Documentation : Documentation explaining the project setup, implementation. And instructions on usage and security best practices.

6.7 Team Size

Maximum: 1 member

7 Mini-Project Seven : Voice Assistant

7.1 Description

This project involves developing a voice-activated assistant using Python. The assistant will recognize voice commands, process them, and respond accordingly. It can perform tasks such as opening applications, fetching information from the web, setting reminders, or managing simple tasks through voice interactions.

7.2 Objectives

- Speech Recognition – Convert user speech into text for command processing.
- Voice Response – Use text-to-speech (TTS) to provide spoken feedback.
- Command Execution – Perform predefined tasks like opening apps, searching the web, or retrieving information.
- User-Friendly Interface – Provide a simple and interactive experience.

7.3 Technical Requirements

Software : Python (SpeechRecognition, pyttsx3 for TTS, OpenAI/Google APIs for NLP), VS Code / PyCharm / Git

Hardware : PC with a microphone and speakers

7.4 Prerequisites

- Basic knowledge of Python and API integration
- Familiarity with speech recognition and text-to-speech libraries
- Understanding of command automation in Python

7.5 Difficulty Level

Beginner to Intermediate

7.6 Deliverables

- Final Product: A working voice assistant
- Source Code Repository: Well-documented GitHub repo
- Speech Recognition Implementation: Accurate voice-to-text conversion
- Project Documentation: implementation details , Setup guide, and usage instructions

7.7 Team Size

Maximum: 1 member

8 Mini-Project Eight : Personal budget tracker App

8.1 Description

This project involves developing a mobile application that helps users track their expenses and manage their budget effectively. Users can log income and expenses, categorize transactions, and view spending summaries through an intuitive interface.

8.2 Objectives

- Expense Tracking – Allow users to input and categorize expenses.
- Budget Management – Set monthly or weekly budgets and track spending.
- Data Visualization – Provide charts and summaries of financial habits.
- Secure Data Storage – Use a local (SQLite) or cloud (Firebase) database.
- User-Friendly Interface – A simple, easy-to-navigate UI.

8.3 Technical Requirements

Software : flutter or react native (for hybrid) — Kotlin or Java (for android)

Hardware : PC for development (and smartphone) for testing

8.4 Prerequisites

- Basic understanding of mobile app development
- Familiarity with databases (SQL or NoSQL)
- Experience with Flutter/Dart or Android development

8.5 Difficulty Level

Intermediate

8.6 Deliverables

- Final Product: A functional budget tracker app
- Source Code Repository: GitHub repo with clean, well-documented code
- Database Integration: Secure and structured data storage
- Project Documentation: Setup guide and user manual

8.7 Team Size

Maximum: 2 members

9 Mini-Project Nine : Blockchain Basics

9.1 Description

This project involves developing a simple blockchain simulation using Python. The system will demonstrate core blockchain concepts such as creating blocks, hashing, linking blocks with cryptographic hashes, and verifying the integrity of the chain. It will be a command-line or basic GUI-based application for visualization.

9.2 Objectives

- Block Creation – Generate blocks containing data and timestamps.
- Cryptographic Hashing – Use hashing (SHA-256) to secure each block.
- Block Linking – Connect blocks through hash references.
- Consensus Simulation – Implement a basic proof-of-work (optional).
- Data Integrity Verification – Ensure blockchain immutability.

9.3 Technical Requirements

Software : Python, Hashlib (for cryptography), Flask (for optional web interface), VS Code / PyCharm / Git

Hardware : PC with standard specifications

9.4 Prerequisites

- Basic understanding of blockchain principles
- Familiarity with Python and data structures
- Knowledge of cryptographic hashing (SHA-256)

9.5 Difficulty Level

Advanced

9.6 Deliverables

- Final Product: A working blockchain simulation
- Source Code Repository: Well-documented GitHub repo
- Blockchain Implementation: Securely linked blocks
- Project Documentation: Setup guide, explanation of implementation and blockchain logic

9.7 Team Size

Maximum: 2 members

10 Mini-Project Ten : AI Image classifier

10.1 Description

This project involves developing an AI-powered image classifier using a neural network. The system will be trained on a dataset to categorize images into predefined classes. The model will utilize deep learning frameworks and can be implemented as a command-line tool or a GUI-based desktop/web application.

10.2 Objectives

- Dataset Processing – Load, preprocess, and augment image data.
- Model Training – Develop and train a neural network for classification.
- Evaluation & Optimization – Test accuracy and fine-tune the model.
- User Interface – Provide a GUI or API for easy image classification.

10.3 Technical Requirements

Software : Python, TensorFlow/Keras or PyTorch, OpenCV (for image processing), Streamlit/Tkinter , VS Code / Git

Hardware : PC with GPU (recommended) or high-performance CPU

10.4 Prerequisites

- Basic knowledge of deep learning and neural networks
- Familiarity with Python and machine learning frameworks
- Understanding of image preprocessing and augmentation

10.5 Difficulty Level

easy to intermediate

10.6 Deliverables

- Final Product: A working AI image classifier
- Source Code Repository: Well-documented GitHub repo
- Trained Model: Optimized for accuracy and performance
- Project Documentation: explanation of implementation , Setup guide, usage instructions, and model architecture details

10.7 Team Size

Maximum: 2 members

11 Mini-Project Eleven : Interactive story game

11.1 Description

This project involves developing a text-based adventure game where players navigate through a branching storyline based on their choices. The game will feature multiple paths, interactive decision-making, and various endings. Implemented with a simple GUI.

11.2 Objectives

- Storyline Development – Design an engaging, branching narrative.
- Game Logic – Implement decision-based progression and multiple endings.
- User Interface – Create a text-based interface (GUI with Tkinter).
- Save & Load System (Optional) – Allow players to save and resume progress.

11.3 Technical Requirements

Software : Python (Tkinter for GUI), JSON (for story data), VS Code / PyCharm / Git

Hardware : PC with standard specifications

11.4 Prerequisites

- Knowledge of Python and basic game development
- Understanding of data structures for story branching
- Experience with handling user input and text-based UI

11.5 Difficulty Level

Intermediate

11.6 Deliverables

- Final Product: A functional interactive story game
- Source Code Repository: Well-documented GitHub repo
- Storyline & Game Logic: Structured and flexible branching narrative
- Project Documentation: implementation details , Setup guide, gameplay instructions, and design choices

11.7 Team Size

Maximum: 2 members

12 Mini-Project Twelve : Data Encryption Tool

12.1 Description

This project involves developing a simple encryption and decryption tool that allows users to securely encode and decode text or files. The tool will implement cryptographic algorithms to protect sensitive data and can be a command-line program or a GUI-based desktop application.

12.2 Objectives

- Encryption & Decryption – Implement secure cryptographic algorithms (AES, RSA, or SHA-256).
- User-Friendly Interface – Provide an intuitive way to encrypt and decrypt data.
- File & Text Encryption – Allow users to encrypt text input or files.
- Security Best Practices – Ensure strong encryption and proper key management.

12.3 Technical Requirements

Software : Python (Tkinter for GUI), Cryptography library, VS Code / Py-Charm / Git

Hardware : PC with standard specifications

12.4 Prerequisites

- Understanding of cryptographic algorithms and key management
- Experience with file handling and data security concepts
- Familiarity with Python and GUI development (in case of developing a GUI)

12.5 Difficulty Level

Easy to Intermediate

12.6 Deliverables

- Final Product: A working encryption & decryption tool
- Source Code Repository: Well-documented GitHub repo
- Encryption Implementation: Secure and efficient encryption methods

- Project Documentation: Instructions on usage, security best practices, and implementation details

12.7 Team Size

Maximum: 1 member

13 Mini-Project Thirteen : Augmented Reality

13.1 Description

This project involves developing a basic augmented reality (AR) application using AR.js, a JavaScript-powered library for web-based AR. The app will allow users to view 3D models or interactive elements overlaid on real-world environments using a camera. It will be accessible through a web browser, requiring no additional software installation.

13.2 Objectives

- Implement AR Features – Use AR.js to overlay digital objects in real-world space.
- Marker-Based or Markerless AR – Display AR content using predefined markers or markerless tracking.
- 3D Model Rendering – Load and interact with basic 3D models (e.g., .glTF or .OBJ files).
- User-Friendly Web Experience – Ensure the app runs smoothly on mobile and desktop browsers.

13.3 Technical Requirements

Software : HTML, JavaScript, AR.js, Three.js (for 3D rendering), Visual Studio Code / Git

Hardware : PC with a webcam, smartphone with a camera (for testing)

13.4 Prerequisites

- Basic knowledge of JavaScript and web development
- Familiarity with AR concepts and 3D rendering (Three.js optional)
- Understanding of how AR.js works with markers and WebXR

13.5 Difficulty Level

Advanced

13.6 Deliverables

- Final Product: A functional AR web app
- Source Code Repository: Well-structured GitHub repo
- AR Implementation: Smooth rendering of 3D models in real-world space
- Project Documentation: Setup instructions, feature explanations, and usage guide

13.7 Team Size

Maximum: 3 members

14 Mini-Project Fourteen : Chat Application

14.1 Description

This project involves developing a mobile chat application that allows real-time messaging between users. The app will use socket programming for communication and provide an intuitive UI for smooth interaction. It will support one-on-one and group chats, with potential features like message history and notifications.

14.2 Objectives

- Real-Time Messaging – Enable instant communication between users.
- Socket Programming – Implement WebSockets for live data exchange.
- User Authentication – Allow users to sign up and log in securely.
- Mobile-Friendly UI – Design an intuitive and responsive user interface.

14.3 Technical Requirements

Software :

- Frontend: Flutter (Dart) / React Native (JavaScript) / Android (Kotlin)
- Backend: Firebase (for authentication & real-time database) / Node.js with WebSockets
- Development Tools: Android Studio / VS Code / Xcode (for iOS)

Hardware : Smartphone (Android or iOS) , PC with an active network connection

14.4 Prerequisites

- Experience with mobile app development (Flutter, React Native, or Android)
- Understanding of WebSockets and socket programming
- Knowledge of Firebase or backend development

14.5 Difficulty Level

Specify whether this project is Beginner, Intermediate, or Advanced.

14.6 Deliverables

- Final Product: A mobile chat app supporting real-time messaging
- Source Code Repository: Well-documented GitHub repo
- Authentication & Security: User login with Firebase or backend authentication
- Project Documentation: Setup guide, API details, and user manual

14.7 Team Size

Maximum: 2 members

15 Mini Project Fifteen : Personal Portfolio Website Development

15.1 Description

Development of a responsive personal portfolio website to showcase professional skills, projects, and experience. The website will serve as a digital identity platform for professional networking and career development.

15.2 Objectives

- Create a mobile-first responsive web design
- Implement interactive elements using JavaScript
- Showcase projects with multimedia content
- Optimize for search engines (SEO)
- Ensure cross-browser compatibility

15.3 Technical Requirements

- First bullet
- Second bullet

15.4 Prerequisites

- HTML5 & CSS3 fundamentals
- Basic JavaScript programming
- Version control with Git
- Understanding of responsive design principles
- Basic UI/UX design concepts

15.5 Difficulty Level

Intermediate

15.6 Deliverables

- Fully functional website deployed on hosting service
- Source code repository (GitHub/GitLab)
- Technical documentation & user manual
- Optimized website assets (compressed images, minified code)

15.7 Team Size

Maximum: 1 member

16 Mini Project Sixteen : Weather Forecast Mobile App Project

16.1 Description

Development of a cross-platform mobile application that provides real-time weather data, forecasts, and alerts. The app will integrate with reliable weather APIs to deliver accurate and up-to-date meteorological information and is optimized for both iOS and Android devices.

16.2 Objectives

- Integrate with a reliable weather API (e.g., OpenWeatherMap, WeatherAPI)
- Design an intuitive and responsive user interface tailored for mobile devices
- Provide real-time weather updates, current conditions, and extended forecasts
- Incorporate location services for automatic weather detection and manual search
- Implement offline capabilities and error handling for improved user experience

16.3 Technical Requirements

Software:	IDE (e.g., VS Code), Git, mobile development framework (e.g., React Native or Flutter)
APIs:	Access to a weather data API (e.g., OpenWeatherMap)
Hardware:	Computer for development, mobile devices (iOS and Android) for testing, Internet connection
Other:	Cross-platform testing tools, free cloud hosting for backend services (e.g., Firebase, Render)

16.4 Prerequisites

- Proficiency in JavaScript/TypeScript (or Dart if using Flutter) and mobile app development frameworks
- Familiarity with API integration and asynchronous programming (AJAX/Fetch)
- Understanding of mobile UI/UX design principles and responsive layouts
- Experience with version control using Git

16.5 Difficulty Level

Intermediate — Requires familiarity with mobile app development, API integration, and cross-platform design principles.

16.6 Deliverables

- **Deployed Mobile Application:** A fully functional weather forecast mobile app, distributed as an APK file for Android devices and via TestFlight for iOS devices (beta testing). Detailed installation instructions will be provided.
- **Source Code Repository:** A well-organized GitHub/GitLab repository with detailed commit history and a comprehensive README.
- **Technical Documentation & User Manual:** Documentation covering app architecture, API integration, deployment steps, and user instructions.
- **Optimized Assets:** Optimized assets including minified code and compressed images for fast load times and smooth performance.

16.7 Team Size

Maximum: 2 members

17 Mini Project Seventeen : Stock Price Predictor

17.1 Description

Develop a stock price prediction system for supermarkets or enterprises. The system will analyze historical stock data, market trends, and external factors to predict future stock prices. This project provides experience in data science, machine learning, and financial modeling.

17.2 Objectives

- Collect and process historical stock price data.
- Implement machine learning models for stock price prediction.
- Integrate real-time market data and external factors.
- Develop a user-friendly dashboard for visualization.
- Deploy the application as a web platform or standalone software.

17.3 Technical Requirements

Software:	• IDE (e.g., Jupyter Notebook, VS Code, PyCharm) • Git for version control • Backend framework (e.g., Flask/Django for Python) • Frontend framework (e.g., React, Vue.js, or plain HTML/CSS/JavaScript) • Database system (e.g., PostgreSQL, Firebase, or Google Sheets API) • Machine learning libraries (e.g., TensorFlow, Scikit-learn, Pandas, NumPy)
Hardware:	Computer for development and testing, Internet connection.
Other:	Cloud hosting service (e.g., AWS, Google Cloud, Heroku) for deployment.

17.4 Prerequisites

- Basic knowledge of Python and machine learning.
- Familiarity with data analysis and financial modeling.
- Understanding of database management and API integration.
- Experience with version control using Git.

17.5 Difficulty Level

Intermediate to Advanced — Suitable for students with knowledge of machine learning and data science looking to apply predictive analytics in real-world scenarios.

17.6 Deliverables

- **Deployed Stock Price Predictor:** A functional system for analyzing and predicting stock prices.
- **Source Code Repository:** A structured repository (e.g., on GitHub) containing all source code, documentation, and deployment instructions.
- **Technical Documentation & User Manual:** Detailed documentation on system architecture, data processing, and predictive modeling techniques.
- **Demonstration Video (Optional):** A short video showcasing the platform's capabilities.

17.7 Team Size

Maximum: 3 members

18 Mini Project Eighteen : Collaborative Coding Platform

18.1 Description

Develop a collaborative coding platform that allows multiple users to code together in real-time. The platform will feature a web-based code editor, user authentication, and basic project management functionalities. This project aims to provide students with hands-on experience in web development, real-time communication, and backend development.

18.2 Objectives

- Implement a web-based collaborative code editor.
- Enable real-time editing and synchronization between multiple users.
- Develop a user authentication and project management system.
- Support multiple programming languages with syntax highlighting.
- Implement a simple file storage system for project persistence.
- Deploy the platform using a free cloud hosting service.

18.3 Mini Project Nineteen : Technical Requirements

Software:	• IDE (e.g., VS Code) • Git for version control • Web development frameworks (e.g., Node.js/Express for backend; React or plain HTML/CSS/JavaScript for frontend) • Database system (e.g., MongoDB or PostgreSQL) • WebSocket or Firebase for real-time collaboration
Hardware:	Computer for development and testing, Internet connection.
Other:	Free cloud hosting service (e.g., Heroku, Firebase, or Render).

18.4 Prerequisites

- Basic knowledge of web development (HTML, CSS, JavaScript).
- Familiarity with backend programming (e.g., JavaScript with Node.js or Python).
- Understanding of real-time communication using WebSockets.
- Basic experience with databases (SQL/NoSQL) and version control using Git.

18.5 Difficulty Level

Intermediate — Suitable for students with foundational web development skills looking to gain experience in real-time applications.

18.6 Deliverables

- **Deployed Collaborative Coding Platform:** A functional web-based application that allows real-time collaborative coding.
- **Source Code Repository:** A structured repository (e.g., on GitHub) containing all source code, documentation, and deployment instructions.
- **Technical Documentation & User Manual:** Detailed documentation on system architecture, functionalities, and user guidelines.
- **Demonstration Video (Optional):** A short video demonstrating the platform's features.

18.7 Team Size

Maximum: 3 members

19 Mini Project Nineteen : Cloud Storage System

19.1 Description

Develop a cloud storage system that enables users to securely upload, store, and manage their files online. The project will feature a user-friendly web interface paired with a robust backend for handling file storage, basic metadata management, and secure authentication. Designed with students in mind, this project provides hands-on experience in full-stack web development, cloud hosting, and essential data security practices.

19.2 Objectives

- Design and implement a user registration and login system.
- Develop a responsive web interface for file upload, download, and management.
- Create a backend service to handle file storage, retrieval, and deletion.
- Integrate a database for managing user data and file metadata.
- Implement basic security measures such as secure authentication and encrypted file storage.
- Deploy the application using a free cloud hosting platform.

19.3 Technical Requirements

Software:	• IDE (e.g., VS Code) • Git for version control • Web development frameworks (e.g., Node.js/Express for the backend, React or plain HTML/CSS/JavaScript for the frontend) • Database system (e.g., MongoDB or PostgreSQL)
Hardware:	Computer for development and testing, Internet connection.
Other:	Free cloud hosting service (e.g., Heroku or Firebase), and open-source libraries for file handling and security.

19.4 Prerequisites

- Basic knowledge of web development (HTML, CSS, JavaScript).
- Familiarity with a backend programming language (e.g., JavaScript with Node.js, Python, or Rust).

- Understanding of databases and basic SQL/NoSQL operations.
- Basic experience with version control using Git.

19.5 Difficulty Level

Beginner to Intermediate — Suitable for students with foundational web development skills who wish to expand their expertise into full-stack and cloud computing.

19.6 Deliverables

- **Deployed Cloud Storage System:** A fully functional web application that enables user registration, secure file upload/download, and file management.
- **Source Code Repository:** A well-organized repository (e.g., on GitHub) containing all source code, documentation, and deployment instructions.
- **Technical Documentation & User Manual:** Comprehensive documentation detailing system architecture, code explanations, and user guidance.
- **Demonstration Video (Optional):** A short video showcasing the key features and usage of the system.

19.7 Team Size

Maximum: 2 members

20 Mini Project Twenty : Check-In System

20.1 Description

Develop a web-based check-in system that allows users to create events and link them with an existing database, Google Sheets, or Excel data for participant registrations. The system will automate confirmation emails upon registration and facilitate real-time check-ins during the event. This project provides hands-on experience in web development, API integration, and event management automation.

20.2 Objectives

- Enable users to create and manage events.
- Integrate with existing registration data sources (e.g., databases, Google Sheets, Excel files).
- Automate sending confirmation emails upon registration.
- Implement real-time check-in functionality during the event.
- Provide a user-friendly dashboard for event organizers to monitor check-ins.
- Deploy the application as a web platform accessible on multiple devices.

20.3 Technical Requirements

Software:	• IDE (e.g., VS Code, PyCharm) • Git for version control • Backend framework (e.g., Flask/Django for Python, Node.js/Express) • Frontend framework (e.g., React, Vue.js, or plain HTML/CSS/JavaScript) • Database system (e.g., PostgreSQL, Firebase, or Google Sheets API) • Email service (e.g., SMTP, SendGrid, Nodemailer)
Hardware:	Computer for development and testing, Internet connection.
Other:	Cloud hosting service (e.g., Heroku, Firebase, AWS) for deployment.

20.4 Prerequisites

- Basic knowledge of web development (HTML, CSS, JavaScript).
- Familiarity with backend programming (e.g., Python, Node.js).
- Understanding of database management and API integration.
- Experience with email automation and authentication.

20.5 Difficulty Level

Intermediate — Suitable for students with basic web development experience looking to integrate automation and API-driven functionalities.

20.6 Deliverables

- **Deployed Check-In System:** A functional web app allowing event creation, automated emails, and real-time check-ins.
- **Source Code Repository:** A structured repository (e.g., on GitHub) containing all source code, documentation, and deployment instructions.
- **Technical Documentation & User Manual:** Detailed documentation on system architecture, API usage, and user guidance.
- **Demonstration Video (Optional):** A short video showcasing the platform's capabilities.

20.7 Team Size

Maximum: 4 members

21 Mini Project Twenty-One : Alarm Mobile App Project

21.1 Description

Develop a cross-platform alarm mobile application that allows users to set, edit, and manage alarms easily. The app will offer a simple and intuitive interface, custom alarm tones, and snooze functionality. It is built using either React Native or Flutter, making it accessible on both Android and iOS platforms.

21.2 Objectives

- Create a user-friendly interface for setting and managing alarms.
- Implement core functionalities including setting multiple alarms, snooze, and alarm cancellation.
- Integrate custom alarm tones and notification features.
- Ensure compatibility on both Android and iOS devices using React Native or Flutter.
- Provide clear error handling and feedback to enhance the user experience.

21.3 Technical Requirements

Software:	IDE (e.g., VS Code), Git, and either React Native or Flutter framework.
Hardware:	Computer for development, mobile devices (Android and/or iOS) for testing, Internet connection.
Other:	Free mobile emulators for testing, access to free libraries for notifications and audio playback.

21.4 Prerequisites

- Basic knowledge of JavaScript/TypeScript (for React Native) or Dart (for Flutter).
- Familiarity with mobile app development concepts and user interface design.
- Experience with version control using Git.
- Understanding of basic mobile app functionalities and event handling.

21.5 Difficulty Level

Beginner to Intermediate

21.6 Deliverables

- **Deployed Mobile Application:** A fully functional alarm mobile app, distributed as a free APK file for Android devices and via TestFlight for iOS devices (beta testing). Detailed installation instructions will be provided.
- **Source Code Repository:** A well-organized GitHub/GitLab repository with clear commit history and a comprehensive README that explains setup and usage.
- **Technical Documentation & User Manual:** Documentation covering app architecture, core functionalities, setup steps, and user instructions.
- **Optimized Assets:** Clean and optimized code, with appropriately sized images and assets ensuring a smooth user experience.

21.7 Team Size

Maximum: 2 members

22 Mini Project Twenty-Two : AI Virtual Assistant for Scheduling

22.1 Description

Develop an AI-powered virtual assistant that helps users schedule their tasks and events by understanding natural language prompts. The assistant will leverage a large language model (LLM) to process user inputs like *“I have leg day at 9 PM tomorrow”* and manage schedules efficiently. This project provides hands-on experience in AI, NLP, and automation.

22.2 Objectives

- Implement natural language processing to interpret user inputs.
- Develop an intelligent scheduling system that organizes tasks automatically.
- Enable reminders and notifications for upcoming events.
- Provide an interactive chatbot interface for seamless user experience.
- Integrate with calendar services (e.g., Google Calendar, Outlook).
- Deploy the assistant as a web or mobile application.

22.3 Technical Requirements

Software:	• IDE (e.g., VS Code, PyCharm) • Git for version control • Backend framework (e.g., Flask/Django for Python, Node.js/Express) • Database system (e.g., PostgreSQL, Firebase) • NLP models (e.g., OpenAI GPT, Google Dialogflow, Rasa)
Hardware:	Computer for development and testing, Internet connection.
Other:	Cloud hosting service (e.g., Heroku, Firebase, AWS) for deployment.

22.4 Prerequisites

- Basic knowledge of Python or JavaScript for backend development.
- Familiarity with machine learning concepts and NLP.
- Understanding of API integration (e.g., Google Calendar API).
- Experience with version control using Git.

22.5 Difficulty Level

Intermediate to Advanced — Suitable for students with programming experience looking to explore AI and automation.

22.6 Deliverables

- **Deployed AI Virtual Assistant:** A functional AI-based scheduling assistant that interprets natural language inputs.
- **Source Code Repository:** A structured repository (e.g., on GitHub) containing all source code, documentation, and deployment instructions.
- **Technical Documentation & User Manual:** Detailed documentation on system architecture, AI model usage, and user guidelines.
- **Demonstration Video (Optional):** A short video showcasing the assistant's capabilities.

22.7 Team Size

Maximum: 3 members

23 Mini Project Twenty-Three : AI for Language Translation

23.1 Description

Develop an AI-driven translation system focused on translating Algerian Daridja or Tamazight. The system will leverage Natural Language Processing (NLP) and machine learning techniques to translate text between these languages and other major languages (e.g., Arabic, French, English). This project aims to support language preservation and enhance communication, making it especially suitable for academic and student environments.

23.2 Objectives

- Build a translation model that handles Algerian Daridja or Tamazight.
- Collect and preprocess a dataset containing texts in the target language(s).
- Experiment with NLP techniques (such as sequence-to-sequence models or transformer-based architectures) to achieve effective translation.
- Evaluate model performance using standard metrics and refine the approach based on feedback.
- Develop a simple user interface (web or mobile) to allow users to test and interact with the translation system.

23.3 Technical Requirements

Software:	Python (3.7+), Jupyter Notebook, Git, TensorFlow or PyTorch, HuggingFace Transformers library
Hardware:	A computer (GPU recommended for training), Internet connection
Other:	Access to open-source NLP datasets, free cloud resources (e.g., Google Colab) for model training and experimentation

23.4 Prerequisites

- Basic proficiency in Python programming and machine learning fundamentals.
- Familiarity with Natural Language Processing (NLP) concepts and libraries.
- Understanding of deep learning frameworks (e.g., TensorFlow or PyTorch).
- Experience with version control (e.g., Git).
- Web or mobile development experience to create a simple user interface.

23.5 Difficulty Level

Intermediate to Advanced

23.6 Deliverables

- **Translation System:** A working AI translation model capable of translating texts between Algerian Daridja (or Tamazight) and other selected languages.
- **Source Code Repository:** A well-documented repository (e.g., on GitHub or GitLab) containing all code, data preprocessing scripts, and model training routines.
- **Dataset Documentation:** A report detailing the dataset collection process, preprocessing steps, and annotation guidelines.
- **User Interface:** A basic web or mobile interface that allows users to input text and receive translations.
- **Technical Documentation & User Manual:** Detailed documentation explaining the system architecture, model performance metrics, and instructions for use.

23.7 Team Size

Maximum: 3 members

24 Mini Project Twenty-Four : 3D Interactive Web Showcase

24.1 Description

Develop an interactive 3D web experience that showcases the club's projects, events, or facilities. This immersive portal leverages modern 3D graphics technologies to create a visually engaging and dynamic presentation, helping to boost the club's online presence and attract interest from current and prospective members.

24.2 Objectives

- Create an interactive 3D environment using WebGL-based libraries such as Three.js or Babylon.js.
- Design and integrate 3D models representing key aspects of the club (e.g., event highlights, club achievements, or facility tours).
- Develop smooth navigation and interactive elements to enhance user engagement.
- Seamlessly integrate the 3D showcase into the club's existing website or create a dedicated web portal.
- Optimize performance to ensure compatibility across different devices and browsers.

24.3 Technical Requirements

Software:	Code editor (e.g., VS Code), Git, modern web browsers, Three.js or Babylon.js library, HTML5, CSS3, JavaScript.
Hardware:	Computer for development and testing, Internet connection.
Other:	Access to 3D modeling tools (e.g., Blender, SketchUp) for creating or editing 3D assets.

24.4 Prerequisites

- Basic knowledge of HTML, CSS, and JavaScript.
- Familiarity with 3D graphics concepts and WebGL fundamentals.
- Understanding of version control (e.g., Git).
- Interest in 3D modeling and interactive design.

24.5 Difficulty Level

Beginner to Intermediate — Suitable for students with foundational web development skills who wish to explore 3D graphics.

24.6 Deliverables

- **Interactive 3D Web Experience:** A fully functional web portal featuring an immersive 3D environment that showcases the club's key projects, events, or facilities.
- **Source Code Repository:** A well-documented repository (e.g., on GitHub/GitLab) containing all source code, assets, and integration instructions.
- **3D Assets and Models:** Original or sourced 3D models and textures used in the project, along with documentation on their creation or attribution.
- **User Documentation:** A manual explaining navigation, interactive features, and guidelines for integrating the 3D showcase with the club's website.
- **Technical Documentation:** Detailed documentation on the project architecture, technologies used, and performance optimization techniques.

24.7 Team Size

Maximum: 2 members